

Perfeito 🔥 — usar **Docker** para um projeto Django é uma excelente prática, especialmente para garantir que todo o ambiente (dependências, banco de dados, etc.) funcione da mesma forma em qualquer máquina.

Vamos passo a passo sobre **como configurar o Docker para um projeto Django em Linux**:



1. Pré-requisitos

Verifique se você tem instalados:

```
docker --version
docker compose version
```

Se não tiver, instale:

```
sudo apt update
sudo apt install docker.io docker-compose -y
sudo systemctl enable --now docker
sudo usermod -aG docker $USER
```

(Depois disso, **faça logout e login novamente** para aplicar o grupo docker.)



2. Estrutura típica do projeto

Suponha que seu projeto Django tenha esta estrutura:

```
meu_projeto/
├── manage.py
├── requirements.txt
├── app_escola/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   └── ...
├── Dockerfile
└── docker-compose.yml
```



3. Criando o Dockerfile

Crie o arquivo `Dockerfile` na raiz do projeto:

```
# Imagem base
FROM python:3.12-slim

# Define o diretório de trabalho
WORKDIR /app

# Copia o arquivo de dependências
COPY requirements.txt .

# Instala as dependências
RUN pip install --no-cache-dir -r requirements.txt
```

```
# Copia o restante do código
COPY . .

# Expõe a porta padrão do Django
EXPOSE 8000

# Comando padrão
CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```



4. Criando o docker-compose.yml

Esse arquivo orquestra o Django + banco de dados (ex: PostgreSQL):

```
version: '3.9'

services:
  web:
    build: .
    container_name: django_app
    command: python manage.py runserver 0.0.0.0:8000
    volumes:
      - ./app
    ports:
      - "8000:8000"
    depends_on:
      - db
    environment:
      - DEBUG=1
      - DB_NAME=escola_db
      - DB_USER=escola_user
      - DB_PASSWORD=senha123
      - DB_HOST=db
      - DB_PORT=5432

  db:
    image: postgres:16
    container_name: postgres_db
    environment:
      - POSTGRES_DB=escola_db
      - POSTGRES_USER=escola_user
      - POSTGRES_PASSWORD=senha123
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"

volumes:
  postgres_data:
```



5. Ajustando o settings.py do Django

No seu settings.py, altere a configuração do banco de dados para usar as variáveis do container:

```
import os
```

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': os.environ.get('DB_NAME', 'escola_db'),
        'USER': os.environ.get('DB_USER', 'escola_user'),
        'PASSWORD': os.environ.get('DB_PASSWORD', 'senha123'),
        'HOST': os.environ.get('DB_HOST', 'localhost'),
        'PORT': os.environ.get('DB_PORT', '5432'),
    }
}
```



6. Subindo os containers

Agora, basta rodar:

```
docker compose up --build
```

Depois, acesse no navegador:

👉 <http://localhost:8000>



7. Comandos úteis

Rodar migrações:

```
docker compose exec web python manage.py migrate
```

Criar superusuário:

```
docker compose exec web python manage.py createsuperuser
```

Entrar no shell do container:

```
docker compose exec web bash
```

Parar os serviços:

```
docker compose down
```



Dica Extra — Arquivo .env

Para não expor senhas no `docker-compose.yml`, você pode criar um arquivo `.env`:

```
DEBUG=1
DB_NAME=escola_db
DB_USER=escola_user
DB_PASSWORD=senha123
DB_HOST=db
DB_PORT=5432
```

E então, no `docker-compose.yml`, substituir:

```
environment:
  - DEBUG=${DEBUG}
  - DB_NAME=${DB_NAME}
  - DB_USER=${DB_USER}
  - DB_PASSWORD=${DB_PASSWORD}
  - DB_HOST=${DB_HOST}
  - DB_PORT=${DB_PORT}
```

Se quiser, posso te ajudar a **gerar esses arquivos automaticamente** (Dockerfile e docker-compose.yml) com base no nome do seu projeto e do app principal.

Quer que eu faça isso?